

IEEE 830-1998 FORMAT

DOCUMENT 2 OF 5
SRS

Infralytix

Software Requirements Specification

Functional and non-functional requirements for the Infralytix AI-Powered Infrastructure Intelligence Platform, covering all ten Intelligence Modules, external interfaces, performance targets, and design constraints.

Functional Requirements

Non-Functional Requirements

Interface Specs

Use Cases

Traces to: Infralytix PRD v1.0

Version 1.0 · July 2026

Table of Contents

Software Requirements Specification

1	Introduction	03
	Purpose, scope, definitions, references, overview	
2	Overall Description	04
	Product perspective, functions, users, constraints	
3	Use Case Overview	05
	Primary actors and top-level use cases	
4	Functional Requirements — FR1-FR10	06
	One block per Intelligence Module	
5	External Interface Requirements	10
	User, hardware, software, communications	
6	Non-Functional Requirements	11
	Performance, security, reliability, usability, more	
7	Design & Implementation Constraints	13
	Technology, regulatory, and process constraints	
8	Requirements Traceability Matrix	14
	Module → requirement → priority → phase	

1

Introduction

Purpose, scope, definitions & references — IEEE 830 §1

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for **InfraIytx**, an AI-powered Infrastructure Intelligence Platform. It is intended for the engineer(s) implementing the system and serves as the authoritative reference for what the system must do, written to be verifiable and testable, consistent with IEEE 830-1998 conventions.

1.2 Scope

InfraIytx analyzes uploaded repositories, generates deployment configurations, monitors infrastructure, explains logs and failures, estimates multi-cloud cost, scans for security issues, audits repository health, generates DevOps automation artifacts, produces architecture diagrams, and exposes a conversational "AI Engineer" — all coordinated by a multi-agent Orchestrator. This document covers the ten Intelligence Modules described in the PRD (Document 1 of 5) at requirement level.

1.3 Definitions, Acronyms & Abbreviations

Term	Definition
Agent	A specialized AI worker responsible for one Intelligence Module's domain (e.g. Log Intelligence Agent)
Orchestrator	The central routing service that dispatches a request to one or more Agents and merges results
FR / NFR	Functional Requirement / Non-Functional Requirement
RBAC	Role-Based Access Control
SUT	System Under Test
P1 / P2 / P3	Priority: Must-have / Should-have / Nice-to-have

1.4 References

- InfraIytx Vision & Product Requirements Document (PRD), v1.0
- IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications

1.5 Overview

Section 2 describes the product at a high level; Section 3 outlines use cases; Section 4 enumerates functional requirements per module; Section 5 covers external interfaces; Section 6 specifies non-functional requirements; Section 7 lists constraints; Section 8 provides traceability back to the PRD roadmap.

2.1 Product Perspective

Infralytix is a new, self-contained system composed of a React SPA frontend, a FastAPI backend, a PostgreSQL data store, and a set of AI agents backed by the Gemini API. It integrates with external systems — GitHub (OAuth + repo access), Docker daemon/SDK, and cloud provider APIs (AWS, Azure, GCP) — but does not replace them; it sits above them as a reasoning and automation layer.

2.2 Product Functions (Summary)

- Analyze an uploaded or connected repository and report architecture, stack, dependencies, and structure (FR1)
- Generate deployment artifacts: Dockerfile, Compose, CI/CD workflow, env templates (FR2)
- Monitor live infrastructure signals — CPU, memory, disk, containers (FR3)
- Parse and explain logs and tracebacks with root cause and fix (FR4)
- Estimate and optimize multi-cloud costs (FR5)
- Detect security issues — secrets, weak configs, vulnerabilities (FR6)
- Audit repository quality — dead code, duplication, missing docs (FR7)
- Generate CI/CD pipelines and infra documentation (FR8)
- Auto-generate architecture and dependency diagrams (FR9)
- Serve as a conversational AI Engineer across all of the above (FR10)

2.3 User Characteristics

User Class	Technical Level	Primary Needs
Full-stack / Python developer	Intermediate	Guided deployment, plain-language error explanations
DevOps / Cloud engineer	Advanced	Fast root-cause, automation generation, override controls
Student / Learner	Beginner	Explanations over raw output, safe sandboxed actions

2.4 Constraints

- AI reasoning depends on the Gemini API; system must degrade gracefully on API errors or rate limits
- Single-developer build — architecture must favor modularity and clarity over premature horizontal scale
- Docker Desktop / Docker Engine must be reachable for Deployment & Infrastructure Intelligence to function locally

2.5 Assumptions & Dependencies

- Users authenticate via email/password or GitHub OAuth before accessing any module
- Kubernetes and Terraform support, RAG documentation, and local Ollama models are Phase 4 / future scope, not required for v1.0 acceptance
- Cost estimation depends on publicly available AWS/Azure/GCP pricing data, refreshed periodically

3

Use Case Overview

Primary actors and top-level interactions

Actor	Use Case	Trigger
Developer	Upload project for analysis	Drags/selects a repo or zip on the Dashboard
Developer	Ask "why is my deployment failing?"	Pastes an error or points AI Engineer at a failed run
DevOps Engineer	Generate deployment configuration	Requests Dockerfile/Compose/CI-CD from Deployment Intelligence
DevOps Engineer	Monitor live infrastructure	Opens Infrastructure Intelligence dashboard
Cloud Engineer	Estimate cloud cost	Submits infra spec to Cloud Intelligence
Security-conscious user	Scan repo for exposed secrets	Runs Security Intelligence scan on connected repo
Any authenticated user	View auto-generated architecture diagram	Opens Architecture Intelligence tab after a Project Intelligence scan
Orchestrator (system)	Route a natural-language request to the correct Agent(s)	Any AI Engineer query

USE CASE EXAMPLE — UC-04: EXPLAIN A FAILING DEPLOYMENT

Actor: Developer · **Precondition:** A project has been uploaded and analyzed at least once.

Flow: (1) Developer pastes a stack trace or points to a failed CI run → (2) Orchestrator routes to Log Intelligence Agent → (3) Agent parses log, classifies severity, determines root cause → (4) Agent proposes a fix, optionally handing off to Deployment Intelligence Agent to regenerate a corrected Dockerfile → (5) Report Intelligence Agent renders a unified explanation.

Postcondition: User sees root cause, severity, and an actionable fix, with an option to apply the generated correction.

4

Functional Requirements

One requirement block per Intelligence Module — IEEE 830 §3

FR1 — Project Intelligence

ID	Requirement	Priority
FR1.1	System shall accept a repository upload (zip) or GitHub URL for analysis	P1
FR1.2	System shall detect the technology stack (languages, frameworks, package managers) from repo contents	P1
FR1.3	System shall produce a dependency analysis including outdated or vulnerable packages	P2
FR1.4	System shall visualize folder structure and flag architectural improvement suggestions	P2

FR2 — Deployment Intelligence

ID	Requirement	Priority
FR2.1	System shall generate a Dockerfile appropriate to the detected stack	P1
FR2.2	System shall generate a Docker Compose file for multi-service projects	P1
FR2.3	System shall generate a GitHub Actions CI/CD workflow file	P1
FR2.4	System shall generate an environment variable template (.env.example) with inferred keys	P2
FR2.5	System shall produce a human-readable deployment guide alongside generated artifacts	P2

FR3 — Infrastructure Intelligence

ID	Requirement	Priority
FR3.1	System shall poll and display CPU, memory, and disk utilization for the monitored host(s)	P1
FR3.2	System shall list running containers and services with health status	P1
FR3.3	System shall generate an AI-written insight summary when a metric crosses a configurable threshold	P2

FR4 — Log Intelligence

ID	Requirement	Priority
FR4.1	System shall accept pasted or uploaded logs: Python tracebacks, Docker logs, server/Nginx logs, system logs	P1
FR4.2	System shall classify the severity of each detected issue (info/warning/error/critical)	P1
FR4.3	System shall output a plain-language root-cause explanation and a proposed solution	P1

FR5 — Cloud Intelligence

ID	Requirement	Priority
FR5.1	System shall estimate monthly cost for a given infra spec on AWS, Azure, and GCP	P1
FR5.2	System shall surface at least three concrete, ranked cost-optimization suggestions per estimate	P2
FR5.3	System shall allow side-by-side comparison of estimates across the three providers	P2

FR6 — Security Intelligence

ID	Requirement	Priority
FR6.1	System shall scan repository contents for exposed secrets and credentials	P1
FR6.2	System shall flag weak or risky Docker and environment configurations	P1
FR6.3	System shall report known-vulnerability matches against dependency versions	P2

FR7 — Repository Intelligence

ID	Requirement	Priority
FR7.1	System shall detect dead code and duplicate code blocks across a scanned repository	P2
FR7.2	System shall flag oversized files and missing documentation (README, docstrings)	P2
FR7.3	System shall surface architecture issues (circular deps, layering violations)	P3

FR8 — DevOps Intelligence

ID	Requirement	Priority
FR8.1	System shall generate CI/CD pipeline definitions beyond the base workflow (multi-stage, matrix builds)	P2
FR8.2	System shall generate infrastructure documentation from the analyzed project state	P2
FR8.3	System shall scaffold a monitoring setup (metrics + basic alerting config)	P3

FR9 — Architecture Intelligence

ID	Requirement	Priority
FR9.1	System shall auto-generate a system architecture diagram from analyzed project data	P2
FR9.2	System shall render a component/dependency graph using React Flow	P2
FR9.3	System shall generate a deployment diagram reflecting the generated Docker/Compose topology	P3

FR10 — AI Engineer

ID	Requirement	Priority
FR10.1	System shall accept free-form natural-language questions ("Why is deployment failing?", "Optimize my Dockerfile")	P1
FR10.2	Orchestrator shall route each query to the correct specialized Agent(s) based on intent classification	P1
FR10.3	System shall merge multi-agent output into a single coherent response via the Report Intelligence Agent	P2
FR10.4	System shall support explicit commands ("Generate Terraform", "Explain Kubernetes YAML") in addition to open questions	P3

5.1 User Interfaces

- Responsive React 19 + TailwindCSS single-page application; minimum supported viewport 1280×720 for full dashboard, with a simplified mobile layout
- Monaco Editor embedded for viewing/editing generated configuration files in-browser
- React Flow canvas for interactive architecture and dependency diagrams

5.2 Hardware Interfaces

None directly; the system runs on standard cloud VM or container infrastructure. Infrastructure Intelligence reads host-level metrics via the OS (through `psutil`) rather than talking to hardware directly.

5.3 Software Interfaces

External System	Interface Type	Purpose
Gemini API	HTTPS / REST	All AI reasoning, generation, and explanation tasks
GitHub	OAuth 2.0 + REST API	Login and repository access for scanning
Docker Engine	Docker SDK (Unix socket / TCP)	Container & service inspection for Infrastructure Intelligence
AWS / Azure / GCP	Provider SDKs / pricing APIs	Cloud cost estimation inputs
PostgreSQL	SQLAlchemy / TCP	Primary persistence layer

5.4 Communications Interfaces

- All frontend-backend traffic over HTTPS using JSON payloads (Axios client)
- JWT bearer tokens on every authenticated request; refreshed on expiry
- Live Infrastructure Intelligence metrics delivered via periodic polling in v1.0 (WebSocket streaming considered for a future iteration)

6

Non-Functional Requirements

IEEE 830 §3.2 — the qualities the system must have, not just what it does

6.1 Performance

ID	Requirement	Target
NFR-P1	Log Intelligence root-cause response time	< 30s (P95)
NFR-P2	Dashboard initial load time	< 2.5s on broadband
NFR-P3	Infrastructure metric refresh interval	≤ 15s polling cadence
NFR-P4	Concurrent analysis jobs supported per instance	≥ 10 without degradation

6.2 Security

- All authentication via JWT with short-lived access tokens; GitHub OAuth as an alternative login path
- Passwords hashed with bcrypt; never stored or logged in plaintext
- Role-Based Access Control (RBAC) distinguishing at minimum: Owner, Collaborator, Viewer
- All traffic served over HTTPS; secrets detected by Security Intelligence must never be persisted verbatim in logs

6.3 Reliability & Availability

- Core analysis modules (FR1, FR4) shall degrade gracefully — return partial results rather than a hard failure — if the Gemini API is unavailable
- Target uptime for the hosted demo instance: 99% monthly

6.4 Usability

- Every AI-generated explanation must be understandable by an intermediate developer without DevOps specialization (plain language, no unexplained jargon)
- Generated configuration files must be viewable and editable in-app (Monaco Editor) before download

6.5 Maintainability

- Backend organized by domain (api / services / ai_agents / deployment / monitoring / security / github / reports) to keep each Intelligence Module independently testable
- Each Agent shall be independently invocable and unit-testable outside the Orchestrator

6.6 Portability

The entire stack shall run via Docker Compose on any Docker-compatible host (Linux, macOS, Windows via WSL2) without code changes, and shall support both SQLite (development) and PostgreSQL (production) through the same SQLAlchemy models.

7

Design & Implementation Constraints

IEEE 830 §2.5 / §3.3

Constraint	Rationale
Backend must be Python 3.12+ / FastAPI	Async-first framework matching the AI-agent workload; strong typing via Pydantic
Frontend must be React 19 + TypeScript	Type safety across a UI with many dynamic, AI-generated data shapes
AI provider is Gemini API for v1.0	LangChain and local Ollama models are explicitly deferred to "future" per the PRD technology stack
Celery / Redis are "future" scope	v1.0 executes agent tasks synchronously or via lightweight background tasks; a task queue is introduced only if async job volume demands it
Kubernetes / Terraform generation is Phase 4	Sequenced after the core analyze→explain→generate loop is proven in Phases 1-2

OPEN QUESTION — CARRIED TO SDD

Whether the Orchestrator performs synchronous sequential agent calls or a fan-out/fan-in async pattern is a design decision, resolved in the Software Design Document (Document 3 of 5), Section on Multi-Agent Orchestration.

Requirements Traceability Matrix

Module → Roadmap Phase → Priority

Module	Key FRs	Roadmap Phase	Priority
Project Intelligence	FR1.1-FR1.4	Phase 1	P1
Log Intelligence	FR4.1-FR4.3	Phase 1	P1
Deployment Intelligence	FR2.1-FR2.5	Phase 2	P1
Repository Intelligence	FR7.1-FR7.3	Phase 2	P2
Infrastructure Intelligence	FR3.1-FR3.3	Phase 3	P1
Cloud Intelligence	FR5.1-FR5.3	Phase 3	P1
Security Intelligence	FR6.1-FR6.3	Phase 3	P1
Architecture Intelligence	FR9.1-FR9.3	Phase 3	P2
DevOps Intelligence	FR8.1-FR8.3	Phase 4	P2
AI Engineer / Orchestrator	FR10.1-FR10.4	Phase 4	P1

NEXT DOCUMENT IN THIS SUITE

Document 3 — Software Design Document (SDD): architecture, database ER diagram, API contracts, and workflow diagrams that satisfy every requirement above.